AllAboutFood Flutter App Technical Documentation

Samuel Munro

Contents

General Flutter Guidance

To further develop the frontend, Dart and Flutter must be installed on your machine. I recommend using VSCode, as there are many helpful extensions for the languages. Follow the steps listed in this documentation to install Dart: https://dart.dev/get-dart, then follow this documentation to install Flutter: https://docs.flutter.dev/get-started/install.

Flutter UI is built using a tree of Widget objects. There are two main types of Widgets – Stateful and Stateless. Read more about them here: https://api.flutter.dev/flutter/widgets/StatefulWidget-class.html, and here: https://api.flutter.dev/flutter/widgets/StatelessWidget-class.html

Screens, items, and any interactable object are widgets. They are very important.

External packages for Flutter can be easily located and installed using the Flutter marketplace.

Pub Dev: https://pub.dev/

As an example, Shared Preferences (local device storage API) can be installed using the steps listed on the store page: https://pub.dev/packages/shared_preferences/install

AWS Environment Number

This got lost a lot for me, and Rick should have this, but this the AWS account number to login to IAM accounts: 238724053954

Firebase

Another installation you will have to perform is the Firebase SDK and CLI. This will allow you to write Firebase code, publish the web version, and interact with Firebase through the console.

Follow this guide to install the Firebase CLI: https://firebase.google.com/docs/cli#setup_update_cli. It will ask you for a project. Since we already have a project, ensure you are logged into the IVFCRdev@gmail.com account (Contact Rick for credentials) and use the existing project that belongs to that account.

Now, I think at this point you should be good to go since AllAboutFood has already been set up with the Firebase SDK. If not, follow these steps to configure it:
https://firebase.google.com/docs/flutter/setup?platform=ios

To publish a version of the app on the web via Firebase hosting, use these 2 commands once you've navigated to your Flutter project directory. That location should look something like this:
...\InteractiveVoiceForCookingRecipes\interactive_voice_for_cooking_recipes

Run '`flutter build web`'

Then, run '`firebase deploy --only hosting:allaboutfoodapp`'

Services

Recipe Data Service

The Recipe Data Service handles all interactions with recipes to and from that database. What follows is a code walkthrough of the service.

Add/Edit Recipe

Add and edit operations are handled through a post request to the associated lambda function. The function URL is defined in the NetworkingConstants class in the app_constants.dart file. The code for the add/edit API call is:

```
Future<Response> addRecipeToDatabase(Recipe r) async {
    final Response response = await _client?.post(
          Uri.parse(
            _addRecipeApiUrl,
          ),
          headers: <String, String>{
            'Content-Type': 'application/json; charset=UTF-8',
          },
          body: await _constructRecipeDatabaseObject(r),
        ) ??
        Response('', 500);

    if (response.statusCode != HttpStatus.ok) {
      throw Exception('Failed to add recipe');
    }

    return response;
  }
```

The edit function is identical code, but is written separately for readability.

The _constructRecipeDatabaseObject(r) call is to a function that packages the Recipe from our system into a JSON body for the post request. Inside this function, control characters are escaped, amounts are set to -1 if they don't exist, and data is combined to form the Strings that are sent to the database. This function is located on line 211 of the recipe_data_service.dart file.

Delete Recipe

The delete recipe function is similar to the add function, but just calls a separate lambda function. In the body of this function, the UUID of the recipe is passed through as well as the authentication token of the user that owns the recipe.

```
Future<Response> deleteRecipeFromDatabase(Recipe r) async {
    final Response response = await _client?.post(
          Uri.parse(
            _deleteRecipeApiUrl,
          ),
          headers: <String, String>{
```

```
            'Content-Type': 'application/json; charset=UTF-8',
          },
          body: '{"${AppText.uuidKey}": "${r.uid}", "Token":"${await
GetIt.I<AuthService>().getAuthToken()}"}',
        ) ??
        Response('', 500);

    if (response.statusCode != HttpStatus.ok) {
      throw Exception('Failed to delete recipe');
    }

    return response;
  }
```

Get All Recipes

This call asks for all recipes in the database and parses them into a format that our system can understand. It passes only an authentication token to the associated lambda function. After the data is received from the database, it is decoded to ensure that non-ascii characters are parsed correctly, and then converted to Recipe objects.

```
  Future<List<Recipe>> getAllRecipes() async {
    List<Recipe> recipes = [];
    final Response getResponse = await _client?.post(
          Uri.parse(
            _getRecipeApiUrl,
          ),
          headers: <String, String>{
            'Content-Type': 'application/json; charset=UTF-8',
          },
          body: '{"Token":"${await GetIt.I<AuthService>().getAuthToken()}"}',
        ) ??
        Response('', 500);

    if (getResponse.statusCode == HttpStatus.ok) {
      final recipeJson = getResponse.body.codeUnits;

      final decoded = json.decode(const Utf8Decoder().convert(recipeJson));
      for (Map recipe in decoded) {
        Recipe toAdd = _constructRecipeDataObject(recipe);
        recipes.add(toAdd);
      }
    }

    return recipes;
  }
```

The _constructRecipeDataObject(Map r) call converts the Map received from the database to a Recipe object for use internal to the app. This function is located on line 133.

Image uploading

This function uploads an image to the Amazon S3 bucket. It first converts the image to a byte stream and then initiates a PUT request to our API. The image is uploaded with the same name as the Recipe UUID (described below). This allows us to generate the link to the image and save it to the database. Recipe image links are referenced in this format: **https://recipephotosrick.s3.amazonaws.com/$uid,** where the

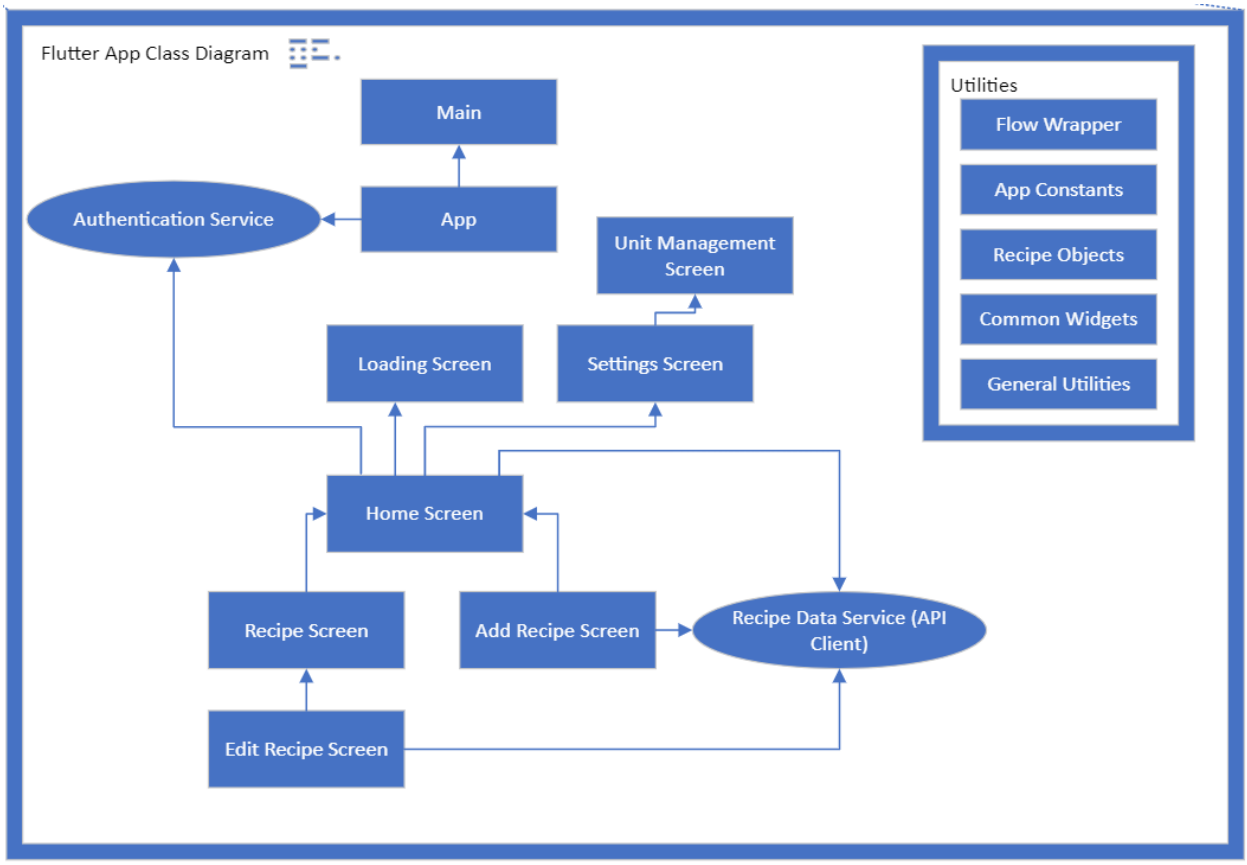
$uid is the UUID of the recipe. Images are updated/uploaded using the link found in this code block, with the specific UUID needed again at the end of the endpoint.

```
Future<Response> uploadImageToDatabase(XFile? image, String uid) async {
    Uint8List imageRep = await image?.readAsBytes() ?? Uint8List(0);
    final Response response = await _client?.put(
          Uri.parse(
            'https://r2addxo0ji.execute-api.us-east-1.amazonaws.com/v1/recipephotosrick/$uid',
          ),
          headers: <String, String>{},
          body: imageRep,
        ) ??
        Response('', 500);

    if (response.statusCode != HttpStatus.ok) {
      throw Exception('Failed to upload image');
    }

    return response;
  }
```

Authentication Service

Inside this file are the functions for sign in, sign out, update display name, verify email, and grabbing the authentication token for use in the recipe service. There are some skeletons of the Google and Facebook login methods as well, but these are not used. The functions closely mirror the Firebase Flutter documentation, located here: https://firebase.google.com/docs/auth/flutter/start

App Flow



When the app opens, a couple of things happen to load the correct data from the database. The home page opens, and two FutureBuilders are constructed. Find out more about FutureBuilders here: https://api.flutter.dev/flutter/widgets/FutureBuilder-class.html

The first FutureBuilder to run is the authentication check. This references the Future in auth_service.dart that returns a User object. If the user is logged in, their data is stored for future use. If not, no user is returned. There is logic further down that shows content based on if there is a user logged in. For example, the buttons on the home screen change. This code describes a button being shown if the loginSnapshot (user data) is not empty.

```
  if (loginSnapshot.data?.isNotEmpty ?? false)
    HomeButton(
        onPressed: () => FlowWrapper.push(
        context,
        AddRecipeScreen(
           onAddRecipe: (r) async {
           FlowWrapper.push(context, const LoadingScreen(message: AppText.loadingRecipes));
           await GetIt.I<RecipeDataService>().addRecipeToDatabase(r).whenComplete(() {
                Navigator.of(context).popUntil(
                       (route) => route.isFirst,
                 );
                 FlowWrapper.pushNoTransition(context, const SearchScreen());
            });

           setState(() {});
           },
           showSnackBar: (String s) => ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:
Text(s))),
                   recipes: snapshot.data ?? [],
           ),
        ),
        displayText: AppText.addRecipe,
        assetImage: const AssetImage('assets/images/addRecipe.jpg'),
   )
```

The second FutureBuilder loads the recipes from the database. It refers to the getAllRecipes() function that was described earlier, which is a Future that returns a List of Recipes.

Once these two Futures have returned their respective data, page updates to show the content of the home screen. Now, all data has been loaded into the app, it can be passed around to the rest of the pages. The recipe list, for example, can be passed to the search screen. Now, we do often refresh this list to ensure that the user's experience is up-to-date while using the app. You can see the same two FutureBuilders on the Search Screen for instance.

Moving Screens Around

Flutter uses a native tool called the Navigator to move screens (widgets) around. I have written a wrapper for this class to make it super easy to do so. My wrapper is called "FlowWrapper" and has 2 useful functions.

1. push(BuildContext c, Widget toPush): this function pushes a page to the screen with default settings.
2. pushNoTransition(BuildContext c, Widget toPush): this pushes a new screen to the page with no animations. Looks clean in certain cases.

If you want to go back a screen, use: `Navigator.of(context).pop();`

Recipe Objects

Recipes are internally represented by Recipe, Step, and Ingredient Objects. The code for these objects can be found in recipe_objects.dart.

Recipe

Recipe instance fields:

- UID (String): Uniquely identifies each recipe.
- Title (String): Title of Recipe.
- Steps (List<RecipeStep>): list of steps, steps are their own objects (see below).
- Ingredients (List<RecipeIngredient>): list of ingredients, ingredients are their own objects (see below).
- Cook Time (Duration*)
- Active Time (Duration*)
- Preparation Time (Duration*)
- Total Time (Duration*)
- Public (Boolean): describes whether a recipe is public or not.
- Owner ID (String): Firebase ID of the owner of this recipe.
- Type (String): Describes the type of the recipe. A list of types can be found in app_constants.dart on line 151.
- Characteristics (List<String>): A list of recipe characteristics. A list of characteristics can be found in app_constants.dart on line 166. This list can be expanded in code and will update in the app automatically.
- Description (String): long field to describe a recipe.
- Summary (String): searchable and shown on search screen.
- Video Link (String): if populated, the app will allow the user to open in an external app such as YouTube.
- Links (List<String>): This list consists of other recipe UUIDs, and allows the user to navigate to other recipes from this recipe.
- Pairs With (List<String>): Purely text that informs users of the recipe of what other items go with this recipe.
- Image URL (String): Allows the system to reference the internet to display an image of a recipe.
- Source (String): clickable URL for the source of the recipe.
- Nutrition (String)
- Equipment (String)
- Servings (double): amount of servings this recipe contains.

*Duration is an internal Dart object. Read more about it here: https://api.dart.dev/be/180361/dart-core/Duration-class.html

Recipe Step

Recipe Step instance field: Description (String): text for the direction/step.

Recipe Ingredient

Recipe Ingredient instance fields:

- Ingredient (String): name of ingredient.
- Amount (double?): amount of ingredient. Can be null. If null, it will be passed to the database as -1.
- Display Amount (String): Used with fractions and describes what is displayed as the amount.
- Unit (String)

API URLs

These URLs are used in the app to communicate with the backend services. These are hosted on Rick's environment and can be found in their respective services on the AWS console.

Add/Edit Lambda function: https://ifw2lmeqjsupk7jkgrowkqaxgi0thayh.lambda-url.us-east-1.on.aws/

Get All Recipes Lambda function: https://j4l67grmk5ptgcdgkvzrzl53nu0yvgxh.lambda-url.us-east-1.on.aws/

Delete Recipe Lambda function: https://uofhq4w72amatdizjvcotrkevq0hxsho.lambda-url.us-east-1.on.aws/

Upload to Amazon S3 Bucket: https://r2addxo0ji.execute-api.us-east-1.amazonaws.com/v1/recipephotosrick/

Get Amazon S3 Bucket resource: https://recipephotosrick.s3.amazonaws.com/

NOTE: add the UUID of the recipe at the end of the bucket URLs to access/upload specific resources. Reference code for this. Furthermore, the Amazon S3 bucket is public, but images can only be displayed with CORS for the **PUBLIC WEB VERSION ONLY** (configured in S3), so images may not show up when running the web version of the app locally (with localhost).

Android Multidex

This isn't a problem I anticipate you will run into, but its pretty niche and tripped me up the first time I ran into it. Pretty much if an Android app gets too big, some extra configuration must be done. This is already done for AllAboutFood. Inside the android/app/build.gradle file you will need to add 2 lines:

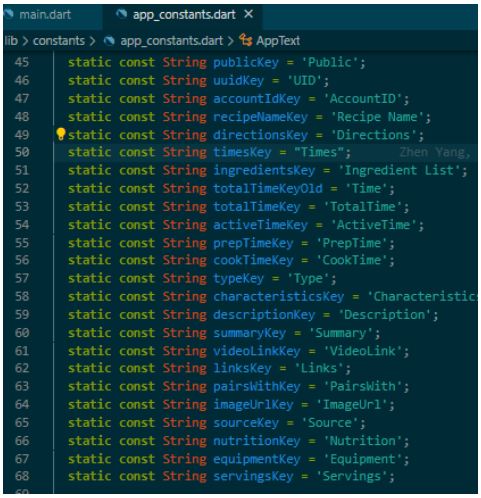
1. `multiDexEnabled true` in the default_config body
2. `implementation 'com.android.support:multidex:1.0.3'` in the dependencies body.

Reference the specific file to see for yourself. Again, I doubt you will run into this and I would not recommend changing this unless it tells you to.

Adding Fields to Recipes

At some point, you may need to add new fields to Recipes for the entire system. Reference the Lambda function documentation for that side of the system. To do so in the app, some files need to be changed.

1. Add a new Key into app_constants.dart. This MUST be the same string used in the lambda functions and will become the attribute name in the database. Pictured are the current keys:



2. Add the new attribute to the Recipe Object definition (described above) and the copyWith function located in that object. This isn't hard and will error once the new field is added to the object.
3. Add the new field to the database object reader and writer in the Recipe Data Service class.
    a. Reader: the function is called: _constructRecipeDataObject(Map r). This function takes database objects and parses them into Recipe objects. Use the existing items as reference and make sure you have a default value if something doesn't exist (common for noSQL database objects). For example, the

    ```
    String title = r[AppText.recipeNameKey] ?? '';
    ```

    has a default title of the empty string. If you don't set defaults then bad things will happen.
    b. Writer: the function called _constructRecipeDatabaseObject(Recipe r) takes a Recipe Object and converts it to something that the database can interpret. Look at the existing code for reference. **THIS CODE WILL NOT ERROR LIKE THE OTHER FUNCTION SO BE SURE NOT TO SKIP THIS STEP, OTHERWISE YOUR NEW FIELD WILL NOT BE WRITTEN TO THE DATABASE**.
4. Fix other errors by adding controllers and text fields within the add_recipe and edit_recipe screens. Reference the existing code here. Be creative, as you will discover some fields are simple text fields while others are more complicated.

Build an APK for the App for Android

With Android devices, you can sideload an APK to an external device. This isn't necessary if you are debugging straight from VSCode, but Rick will need APK updates for his devices, and until the app is published on the Play Store this is the way to do it. Go to the project location (...\InteractiveVoiceForCookingRecipes\interactive_voice_for_cooking_recipes). Generate the APK using this command: `flutter build apk`

This will produce an APK file. It will tell you where it is located. To install,

1. Have the APK on a computer you can plug your phone/tablet into.
2. Plug your phone/tablet into your computer and get it so you can view the files of the android device in your windows file explorer.
3. Drag the downloaded file from your computer into the documents folder of your device (for me that is located at: SamsAndroid\Internal Storage\Documents
4. Go onto your device and locate the copied file. Click on it. My phone prompted me to go to settings to allow the files app to install other apps on the device. Allow this if applicable.
5. From here, the app should install and ask you to open it.